

编程基础 V — 函数式编程范式

软件工程与计算 I · 2026 · 南京大学

刘钦

Outline

- 避免重复 (DRY)
- 函数式编程范式
 - 过程抽象 (高阶函数)
 - 数据抽象 (有序对)
 - 符号求导
 - 元编程 (同像性 · 宏 · Eval)
 - 信号流 (管道范式)
 - Java 8 Stream API
 - 函数式编程特点
 - Hy
- 编程的现实考量
- 证明程序的正确性
 - Dijkstra 不变量法
 - Hoare 公理 (三元组 · D0—D3)
- 习题课
 - 递归与迭代
 - Lambda 演算
 - 过程建模
- 【AI视角】: JAX · LLM 规格验证
- 总结: 核心概念 · 思维转变 · 知识延伸

避免重复

Don't Repeat Yourself

Ctrl C / Ctrl V 的诱惑

当需要相似代码时，最简单的做法是复制粘贴。

```
System.out.println("ctrl c");  
System.out.println("ctrl c");  
System.out.println("ctrl c");  
// ... 重复 N 次
```

复制粘贴会带来**维护噩梦**。修改时需要改 N 处，极易遗漏。

消除重复 — 用循环

```
// 重复版本
System.out.println("ctrl c");
System.out.println("ctrl c");
System.out.println("ctrl c");
```

```
// 循环版本
for (int i = 0; i < n; i++) {
    System.out.println("ctrl c");
}
```

循环是消除重复输出的第一步。

重复代码 — 相似函数

```
public double getAveragemarks(List<Student> students) {  
    double total = 0;  
    for (Student s : students) {  
        total += s.getMarks();  
    }  
    return total / students.size();  
}  
  
public double getAverageAge(List<Student> students) {  
    double total = 0;  
    for (Student s : students) {  
        total += s.getAge();  
    }  
    return total / students.size();  
}
```

两个函数结构完全相同，仅取值字段不同。

消除重复 — 过程抽象

```
public double calcAverage(List<Student> students,  
                           Function<Student, Double> getter) {  
    double total = 0;  
    for (Student s : students) {  
        total += getter.apply(s);  
    }  
    return total / students.size();  
}  
  
// 调用  
calcAverage(students, Student::getMarks);  
calcAverage(students, Student::getAge);
```

将变化的部分（取值字段）抽象为参数，消除重复结构。

重复代码 — if-for 混合

```
def logo_play(commands):  
    for cmd in commands:  
        if cmd == 'forward':  
            turtle.forward(100)  
        if cmd == 'left':  
            turtle.left(90)  
        if cmd == 'right':  
            turtle.right(90)  
        if cmd == 'backward':  
            turtle.backward(100)  
        # ... 更多 if 分支
```

每增加一条命令，就要新增一个 `if` 分支。

消除重复 — 数据驱动分发

```
COMMAND_MAP = {  
    'forward': lambda: turtle.forward(100),  
    'left':    lambda: turtle.left(90),  
    'right':   lambda: turtle.right(90),  
    'backward': lambda: turtle.backward(100),  
}  
  
def logo_play(commands):  
    for cmd in commands:  
        if cmd in COMMAND_MAP:  
            COMMAND_MAP[cmd]()
```

优点:

- 增加命令只需往字典里加一行
- 逻辑与数据分离
- 消除重复的 `if` 结构

消除重复 — 数据过滤

```
// 原始版本
public double getTotalSum(List<Order> orders) {
    double total = 0;
    for (Order o : orders) {
        if (o.getStatus() != CANCELLED) {
            total += o.getAmount();
        }
    }
    return total;
}
```

```
// Stream 版本
public double getTotalSum(List<Order> orders) {
    return orders.stream()
        .filter(o -> o.getStatus()
            != CANCELLED)
        .mapToDouble(Order::getAmount)
        .sum();
}
```

过程抽象（高阶函数）

Higher-Order Functions

高阶函数

高阶函数也是一种抽象——过程作为参数。

将计算的**模式**从具体过程中提取出来，用函数参数表示变化的部分。

- 函数可以作为**参数**传入另一个函数
- 函数可以作为**返回值**
- 这样的函数称为**高阶函数**（Higher-Order Functions）

案例：三个相似的求和过程

$$\text{sum-integers}(a, b) = \sum_{k=a}^b k$$

```
; 计算从 a 到 b 的整数之和
(define (sum-integers a b)
  (if (> a b) 0
      (+ a (sum-integers (+ a 1) b))))
```

$$\text{sum-cubes}(a, b) = \sum_{k=a}^b k^3$$

```
; 计算从 a 到 b 的整数立方之和
(define (sum-cubes a b)
  (if (> a b) 0
      (+ (cube a) (sum-cubes (+ a 1) b))))
```

$$\text{sum-pi}(a) = \sum_{k=0}^{\infty} \frac{1}{(4k+1)(4k+3)} = \frac{\pi}{8}$$

```
; 计算 π 近似级数: 1/(1·3) + 1/(5·7) + 1/(9·11) + ...
(define (sum-pi a b)
  (if (> a b) 0
      (+ (/ 1.0 (* a (+ a 2)))
          (sum-pi (+ a 4) b))))
```

三个函数的共同结构

```
(define (<name> a b)
  (if (> a b)
      0
      (+ (<term> a)
         (<name> (<next> a) b))))
```

- `<term>`：对每项 `a` 施加的变换
- `<next>`：从 `a` 前进到下一项的步骤

将这两个变化点抽象为参数，即得高阶函数 `sum`。

抽象为高阶函数 sum

```
(define (sum term a next b)
  (if (> a b)
      0
      (+ (term a)
          (sum term (next a) next b))))
```

- `term` : 函数参数, 对每项求值
- `next` : 函数参数, 计算下一项的位置

用 sum 重写三个过程

```
(define (sum-integers a b)
  (sum (lambda (x) x)
    a
    (lambda (x) (+ x 1))
    b))
```

```
(define (sum-cubes a b)
  (sum cube
    a
    (lambda (x) (+ x 1))
    b))
```

```
(define (sum-pi a b)
  (sum (lambda (x) (/ 1.0 (* x (+ x 2))))
    a
    (lambda (x) (+ x 4))
    b))
```

lambda 表达式

； 匿名函数

```
(lambda (x) (+ x 4))  
(lambda (x) (/ 1.0 (* x (+ x 2))))
```

； lambda 等价于具名函数定义

```
(define (increment x) (+ x 1))
```

； 等价于

```
(define increment (lambda (x) (+ x 1)))
```

let — 局部变量

; let 定义局部绑定

```
(let ((x 2) (y 3))
```

```
  (* x y))
```

```
; => 6
```

; let 的本质是 lambda 立即调用

```
(let ((x 2) (y 3)) (* x y))
```

; 等价于

```
((lambda (x y) (* x y)) 2 3)
```

let 语法糖：用 lambda 表达局部作用域。

数据抽象

Data Abstraction

有理数

数据抽象：将数据的使用方式与表示方式分离。

用构造器和选择器定义有理数的接口：

```
(define (make-rat n d) (cons n d)) ; 构造
(define (numer x)      (car x))    ; 分子
(define (denom x)      (cdr x))    ; 分母

(define (add-rat x y)
  (make-rat (+ (* (numer x) (denom y))
               (* (numer y) (denom x)))
            (* (denom x) (denom y))))
```

有理数 — 约化

```
; 用 gcd 约分
(define (make-rat n d)
  (let ((g (gcd n d)))
    (cons (/ n g) (/ d g))))

; 测试
(numer (make-rat 6 9))    ; => 2
(denom (make-rat 6 9))    ; => 3
```

实现细节（是否约化）对使用者**完全透明**。

链表

Scheme 用 `cons` 构造序对，用 `car` / `cdr` 解构：

```
(cons 1 (cons 2 (cons 3 nil)))
```

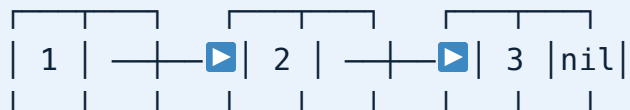
; 等价于

```
(list 1 2 3)
```

; => (1 2 3)

```
(car (list 1 2 3)) ; => 1
```

```
(cdr (list 1 2 3)) ; => (2 3)
```



list-ref

; 取链表第 n 个元素 (从 0 开始)

```
(define (list-ref items n)
  (if (= n 0)
      (car items)
      (list-ref (cdr items) (- n 1))))
```

```
(list-ref (list 1 4 9 16 25) 3)
```

```
; => 16
```

scale-list

; 原始递归版本

```
(define (scale-list items factor)
  (if (null? items)
      nil
      (cons (* (car items) factor)
            (scale-list (cdr items) factor))))
```

; 用 map 简化

```
(define (scale-list items factor)
  (map (lambda (x) (* x factor)) items))
```

map

```
(define (map proc items)
  (if (null? items)
      nil
      (cons (proc (car items))
              (map proc (cdr items)))))
```

; 应用示例

```
(map abs (list -10 2.5 -11.6 17))
```

```
; => (10 2.5 11.6 17)
```

```
(map (lambda (x) (* x x)) (list 1 2 3 4))
```

```
; => (1 4 9 16)
```

map 将变换与遍历分离，是函数式编程的核心原语。

层次性结构（树）

； 嵌套列表构成树

```
(cons (list 1 2) (list 3 4))
```

； => ((1 2) 3 4)

； 等价于：

； *

； /|\

； (1 2) 3 4

； 计算叶节点数

```
(define (count-leaves x)
```

```
  (cond ((null? x) 0)
```

```
        ((not (pair? x)) 1)
```

```
        (else (+ (count-leaves (car x))
```

```
                  (count-leaves (cdr x))))))
```

scale-tree

; 递归版本

```
(define (scale-tree tree factor)
  (cond ((null? tree) null)
        ((not (pair? tree))
         (* tree factor))
        (else
         (cons (scale-tree (car tree) factor)
                 (scale-tree (cdr tree) factor))))))
```

; 用 map 的版本

```
(define (scale-tree tree factor)
  (map (lambda (sub-tree)
        (if (pair? sub-tree)
            (scale-tree sub-tree factor)
            (* sub-tree factor)))
       tree))
```

求导 — 符号微分

接口：(deriv <expr> <var>) 对 <expr> 关于 <var> 求导。

求导规则：

$$\begin{aligned}\frac{d(c)}{dx} &= 0 & \frac{d(x)}{dx} &= 1 \\ \frac{d(u+v)}{dx} &= \frac{du}{dx} + \frac{dv}{dx} \\ \frac{d(u \cdot v)}{dx} &= u \cdot \frac{dv}{dx} + v \cdot \frac{du}{dx}\end{aligned}$$

deriv 实现

```
(define (deriv exp var)
  (cond ((number? exp) 0)
        ((variable? exp)
         (if (same-variable? exp var) 1 0))
        ((sum? exp)
         (make-sum (deriv (addend exp) var)
                     (deriv (augend exp) var)))
        ((product? exp)
         (make-sum
          (make-product (multiplier exp)
                        (deriv (multiplicand exp) var))
          (make-product (deriv (multiplier exp) var)
                        (multiplicand exp))))
        (else (error "unknown expression type" exp))))
```

make-sum 的实现

; 选择器

```
(define (sum? x)      (and (pair? x) (eq? (car x) '+)))  
(define (addend e)    (cadr e))  
(define (augend e)    (caddr e))
```

; 构造器 (化简规则)

```
(define (make-sum a1 a2)  
  (cond ((=number? a1 0) a2)  
        ((=number? a2 0) a1)  
        ((and (number? a1) (number? a2)) (+ a1 a2))  
        (else (list '+ a1 a2))))
```

```
(deriv '(+ x 3) 'x) ; => 1
```

make-product — 练习

思考：如何实现 `make-product` ？

； 选择器

```
(define (product? x)
  (and (pair? x) (eq? (car x) '*)))
(define (multiplier e) (cadr e))
(define (multiplicand e) (caddr e))
```

； 构造器 (请完成化简规则)

```
(define (make-product m1 m2)
  (cond ((or (=number? m1 0) (=number? m2 0)) 0)
        ((=number? m1 1) m2)
        ((=number? m2 1) m1)
        ((and (number? m1) (number? m2)) (* m1 m2))
        (else (list '* m1 m2))))
```

元编程

Code as Data · Macros · Eval

什么是元编程？

三个层次

层次	描述
普通编程	程序操作数据
元编程	程序操作程序（代码）
元元编程	程序操作"操作程序的程序"

常见形式

- 编译器、解释器
- 宏 (Macros)
- 反射 (Reflection)
- 代码生成、Lint 工具

为什么 Lisp 天生适合元编程？

```
； 这是代码（执行加法）
(+ 1 2)    ; => 3

； 加引号：变成数据（列表）
'(+ 1 2)   ; => (+ 1 2)，不执行

； eval：把数据当代码执行
(eval '(+ 1 2)) ; => 3
```

代码就是数据，数据就是代码
——Lisp 的同像性 (Homoiconicity)

同像性 — Code is Data

`deriv` 就是元编程的典型示例：

```
; 数学表达式被表示为普通列表  
(deriv '(* x x) 'x)  
; 输入：列表 (* x x)  
; 操作：把列表当"代码结构"来分析和变换  
; 输出：新列表 (+ (* x 1) (* 1 x)), 即 2x
```

关键点：

- `'(* x x)` 既是 **数学表达式**，也是普通的 **Scheme 列表**
- `car` / `cdr` 就能拆解表达式结构
- `list` / `cons` 就能构造新的表达式

Lisp 不需要专门的 AST 库——列表 **就是** AST。

quote 与 quasiquote

; quote: 完全不求值, 返回原始结构

```
'(1 2 3)           ; => (1 2 3)
```

```
'(+ x y)           ; => (+ x y), x y 不求值
```

; quasiquote (反引号): 模板中用 `,` 插入求值结果

```
(define x 42)
```

```
`(the answer is ,x) ; => (the answer is 42)
```

; unquote-splicing: `,@` 展开列表到模板中

```
(define nums '(1 2 3))
```

```
`(values are ,@nums done)
```

```
; => (values are 1 2 3 done)
```

`quasiquote` 是代码模板——构造宏时的核心工具。

Lisp 宏 (Macros)

函数在运行时处理值；宏在展开期处理代码结构。

```
; 定义 my-and 宏 (实现短路求值)
(define-syntax my-and
  (syntax-rules ()
    ((_) #t)
    ((_ e) e)
    ((_ e1 e2 ...)
     (if e1 (my-and e2 ...) #f))))

; 使用
(my-and #t (= 1 1) (> 3 2)) ; => #t

; 展开结果 (编译期替换):
; (if #t (my-and (= 1 1) (> 3 2)) #f)
; (if #t (if (= 1 1) (my-and (> 3 2)) #f) #f)
; ...
```

宏让你扩展语言本身——`if`、`let`、`and` 在 Lisp 中都可以用宏实现。

my-and 解析 — 三条模式

`syntax-rules` 按**模式匹配**顺序替换代码，`_` 是宏名占位符，`...` 匹配零或多个元素。

模式	展开结果	含义
<code>(_)</code>	<code>#t</code>	无参数：零个条件全为真（同 <code>all([])</code> ）
<code>(_ e)</code>	<code>e</code>	一个参数：直接返回其值（保留非布尔值）
<code>(_ e1 e2 ...)</code>	<code>(if e1 (my-and e2 ...) #f)</code>	多参数：递归展开为嵌套 <code>if</code>

展开过程示例：

```
(my-and A B C)
; → (if A (my-and B C) #f)           ; 命中模式3, e1=A, e2...=B C
; → (if A (if B (my-and C) #f) #f) ; 再次命中模式3
; → (if A (if B C #f) #f)           ; 命中模式2, 直接返回 C
```

my-and 解析 — 为什么必须用宏？

短路求值：B 只在 A 为真时才求值——这是宏的核心价值。

若用普通函数 ❌

```
(define (bad-and a b)
  (if a b #f))

; 调用时，所有参数先求值！
(bad-and #f (launch-missiles!))
;           ↑ 即使 a=#f
;           导弹仍然发射了
```

函数调用：先对所有参数求值，再传入函数体。无法短路。

用宏 ✅

```
(my-and #f (launch-missiles!))

; 宏展开（编译期）：
; → (if #f (my-and (launch-missiles!)) #f)

; 运行时：
; if 条件为 #f，直接返回 #f
; (launch-missiles!) 永远不执行
```

宏把 e2 放进 if 的真值分支，求值权交给运行时的 if，天然短路。

一句话：函数求值后传参，宏变换代码结构——需要控制“何时求值”时，必须用宏。

元编程对比 (1/2) — Java 反射

运行时获取类结构并动态调用——程序在执行期"看见"自己。

```
// 运行时获取类信息，无需编译期知道具体类型
Class<?> cls = Class.forName("java.util.ArrayList");
Method m = cls.getMethod("add", Object.class);
Object list = cls.getDeclaredConstructor().newInstance();
m.invoke(list, "hello");
```

典型应用场景：

框架	反射用途
Spring	扫描 <code>@Autowired</code> ，自动注入依赖
JUnit	发现 <code>@Test</code> 方法，自动执行
Jackson	读取字段名，序列化/反序列化 JSON

代价：运行时类型检查 → 出错时抛 `ClassNotFoundException` / `NoSuchMethodException`。

元编程对比 (2/2) — Lisp 宏

编译期变换代码结构——程序在被编译前就被改写。

```
(define-syntax swap!  
  (syntax-rules ()  
    ((_ a b)  
      (let ((tmp a))  
        (set! a b)  
        (set! b tmp))))))  
  
(swap! x y)  
; 编译期展开为 →  
; (let ((tmp x)) (set! x y) (set! y tmp))
```

维度	Lisp 宏	Java 反射
时机	编译期展开	运行时调用
性能	零运行时开销	有反射开销
错误	编译期可检查	运行时异常
能力	可创造新语法	只能调用已有接口

宏 = 在编译器层面扩展语言；反射 = 在运行时检查程序结构。

Python 装饰器 — 高阶函数的语法糖

```
# 装饰器本质 = 高阶函数
def log(func):
    def wrapper(*args, **kwargs):
        print(f"调用 {func.__name__}, 参数 {args}")
        result = func(*args, **kwargs)
        print(f"返回值: {result}")
        return result
    return wrapper

@log                                # 等价于 add = log(add)
def add(a, b):
    return a + b

add(1, 2)
# 调用 add, 参数 (1, 2)
# 返回值: 3
```

装饰器 = 用函数变换函数——元编程在 Python 中的主要形式。

`@property`、`@staticmethod`、`@dataclass` 都是装饰器。

元循环求值器

用 Scheme 实现 Scheme 解释器——元编程的极致：

```
(define (my-eval exp env)
  (cond
    ((self-evaluating? exp) exp)
    ((variable? exp) (lookup-variable-value exp env))
    ((quoted? exp) (text-of-quotation exp))
    ((lambda? exp) (make-procedure
                     (lambda-parameters exp)
                     (lambda-body exp) env))
    ((application? exp)
     (my-apply
      (my-eval (operator exp) env)
      (list-of-values (operands exp) env)))
    (else (error "未知表达式" exp))))
```

这是 SICP 第四章的核心内容：一门语言的解释器用该语言本身写出。
理解元循环求值器，就理解了“语言是什么”。

my-eval 逐行解析

my-eval 的核心逻辑：判断表达式的类型，分别处理。

cond 分支	匹配什么	做什么
self-evaluating?	数字、字符串等字面量	直接返回自身，无需求值
variable?	符号（变量名）	在环境 env 中查找绑定值
quoted?	(quote x) / 'x	返回被引用的原始结构，不求值
lambda?	(lambda ...)	创建闭包：参数 + 函数体 + 当前环境
application?	(f arg1 arg2 ...)	递归求值函数和参数，再调用 my-apply
else	未知结构	报错

my-eval 解析 — application 分支

最复杂、最核心的分支：**函数调用**。

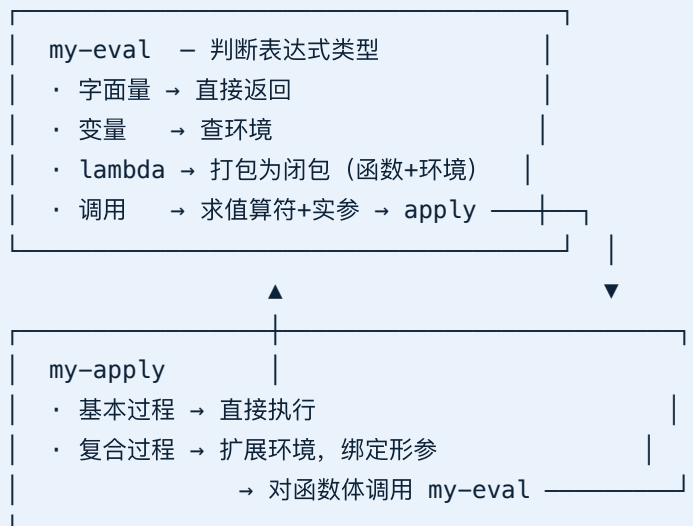
```
((application? exp)
 (my-apply
  (my-eval (operator exp) env)      ; ① 求值函数部分
  (list-of-values (operands exp) env) ; ② 求值所有参数
 ))
```

执行顺序：

```
(my-eval '(+ 1 2) env)
|
├─ operator  → '+' → my-eval → 查 env → 得到加法过程
├─ operands  → (1 2) → list-of-values → (1 2)
|
└─ my-apply  加法过程 '(1 2) → 3
```

my-eval 和 **my-apply** **相互递归**：eval 遇到调用就转给 apply，apply 执行函数体时再调 eval——这对互递归驱动了整个求值引擎。

my-eval 解析 — eval / apply 互递归



两者相互递归：`eval` 遇到函数调用 → `apply`；`apply` 执行函数体 → `eval`。

这对互递归就是几乎所有语言解释器的核心骨架——不超过 50 行，实现一门完整语言。

信号流

枚举器 → 过滤器 → 映射 → 累积器

信号流范式

将计算看作数据流经变换管道：

枚举器 —▶ 过滤器 —▶ 映射器 —▶ 累积器

示例：计算值为奇数的叶节点的平方和

tree leaves —▶ odd? —▶ square —▶ +, 0

1. **枚举：**从树中枚举所有叶节点
2. **过滤：**筛选出奇数叶节点
3. **映射：**对每个奇数叶节点求平方
4. **累积：**将所有平方值求和

sum-odd-squares

```
(define (sum-odd-squares tree)
  (cond ((null? tree) 0)
        ((not (pair? tree))
         (if (odd? tree) (square tree) 0))
        (else (+ (sum-odd-squares (car tree))
                  (sum-odd-squares (cdr tree))))))
```

传统递归写法将枚举、过滤、映射、累积混在一起，难以复用。

sum-odd-squares — 管道版本

用信号流原语将四步清晰分离：

```
; 枚举树的所有叶节点
(define (enumerate-tree tree)
  (cond ((null? tree) nil)
        ((not (pair? tree)) (list tree))
        (else (append (enumerate-tree (car tree))
                        (enumerate-tree (cdr tree))))))

; 管道版 sum-odd-squares
(define (sum-odd-squares tree)
  (accumulate +
              0
              (map square
                   (filter odd?
                          (enumerate-tree tree)))))
```

enumerate-tree —▶ filter odd? —▶ map square —▶ accumulate +

每个步骤单一职责，可独立替换复用。

所有偶数的 Fibonacci 数列

用信号流分解：枚举整数 → 映射为 fib → 过滤偶数 → 收集

```
(define (even-fibs n)
  (define (next k)
    (if (> k n)
        nil
        (let ((f (fib k)))
          (if (even? f)
              (cons f (next (+ k 1)))
              (next (+ k 1))))))
  (next 0))
```

传统写法同样将四步混杂。信号流将它们清晰分离。

Java Stream — 案例背景

创建一个 Messages 集合：

```
List<Message> messages = new ArrayList<>();  
messages.add(new Message("aglover", "foo", 56854));  
messages.add(new Message("aglover", "foo", 85));  
messages.add(new Message("aglover", "bar", 9999));  
messages.add(new Message("rsmith", "foo", 4564));
```

需求：找到所有延迟超过 3000 秒的消息，并计算总延迟。

普通 Java 青年写法

```
long totalWaitTime = 0;
for (Message message : messages) {
    if (message.delay > 3000) {
        totalWaitTime += message.delay;
    }
}
```

文艺 Java 8 青年写法

```
// Java 8 Stream API — 信号流风格
long totWaitTime = messages.stream()
    .filter(m -> m.delay > 3000)
    .mapToLong(m -> m.delay)
    .sum();
```

操作	类型	说明
<code>filter(pred)</code>	中间操作	过滤满足条件的元素
<code>map(f)</code>	中间操作	变换每个元素
<code>mapToLong(f)</code>	中间操作	映射为 long 流
<code>reduce(init, f)</code>	终止操作	累积
<code>sum()</code>	终止操作	求和

函数式编程特点

Functional Programming Paradigm

函数式编程

In computer science, functional programming is a programming paradigm — a style of building the structure and elements of computer programs — that treats computation as the evaluation of mathematical functions and avoids changing-state and mutable data.

— Wikipedia

函数式编程（Functional Programming）将计算机运算看作**数学中函数的计算**，并且避免了**状态**以及**变量**的概念。

函数式编程范式 — 三大特点

A. Functions As First-class Citizens (函数是头等公民)

函数可以作为参数、返回值、赋值给变量，通常以高阶函数形式存在。

B. No Side Effects (无副作用)

副作用：调用函数时，除返回值外还对外部状态产生影响（如修改全局变量）。严格的函数式编程要求函数**无副作用**——返回值唯一取决于参数。

C. No Changing-state (不可变状态)

函数是基础元素，变量不可修改。状态通过**函数参数**保存，而非可变变量。最好的例子是递归。

纯函数示例 — 翻转字符串

```
String reverse(String arg) {  
    if (arg.length() == 0) {  
        return arg;  
    } else {  
        return reverse(arg.substring(1, arg.length()))  
            + arg.substring(0, 1);  
    }  
}
```

- **无副作用**：不修改任何外部状态
- **不可变**：通过递归而非循环变量处理
- **引用透明**：相同输入总返回相同输出

Currying（柯里化）

将接受多个参数的函数转换为接受单一参数并返回新函数的技术。

```
// 普通写法
int pow(int i, int j);
int square(int i) {
    return pow(i, 2);
}
```

```
// 支持 Currying 的语言
square = int pow(int i, 2);
// 直接将 pow 的第二参数固定为 2
```

Currying 是一种快速且简单地实现函数封装的技术。

Concurrency（并发）

由于**无副作用**，函数不依赖全局状态，程序天然支持并发执行。

```
String s1 = somewhatOperation1();  
String s2 = somewhatOperation2();  
String s3 = concatenate(s1, s2);
```

- 函数式语言的编译器可发现 `s1`、`s2` 相互独立，**自动并行执行**
- 不需要锁机制，不存在**死锁或竞争条件**
- 函数式编程非常适合**高度并行**的应用程序

Lazy Evaluation (惰性求值)

惰性求值 (Lazy Evaluation)，又称惰性计算，目的是最小化计算机要做的工作。

- 不会在赋值时立即求值，而是在真正需要时才求值
- 可以省去无谓的计算（如条件分支中未用到的参数）
- 支持构造无限数据结构（如无穷的 Fibonacci 序列）

```
-- Haskell: 无穷 Fibonacci 列表 (惰性求值)
fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
take 10 fibs -- => [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Hy — Python 上的 Lisp 方言

Hy 是运行在 Python 虚拟机上的 Lisp/Scheme 风格语言，连接函数式编程与 Python 生态。

```
; 定义函数
(defn square [x]
  (* x x))

(print (square 5))    ; => 25

; 高阶函数: map + filter
(list (map square (filter odd? [1 2 3 4 5])))
; => [1, 9, 25]
```

- 语法是 **Lisp/Scheme**，运行时是 **Python**
- 可直接调用所有 Python 库
- 支持 lambda、高阶函数、不可变数据等函数式特性

现实考量

Practical Considerations

现实考量

编程面临三类现实约束：

- 复杂性与规模
- 逻辑与物理
- 时间与空间

An Intellectual Challenge

"The art of programming is the art of organising complexity."

— Edsger W. Dijkstra

"Difference in scale is one of the major sources of our difficulties in programming."

— E.W. Dijkstra

An "imperfect" Implementation

逻辑上: `a == b` 且 `b == c` 能推出 `a == c`

但计算机实现是否 perfect?

```
double a = 0.1 + 0.2; // 0.30000000000000004
double b = 0.3;
System.out.println(a == b); // false!
```

浮点数的精度问题: 计算机的物理实现与数学逻辑存在差距。

Proof vs Implementation · Dynamic vs Static

Proof vs Implementation

- 逻辑实现（纸上的数学证明）
- 物理实现（硬件、浮点数、内存限制）

Dynamic vs Static

- Static：代码、存储、数据结构
- Dynamic：执行、处理器、算法

Storage Space vs Computation Speed

- Processor：active, computation speed, fast
- Store：passive, storage space, large

案例 1：想要得到 $f(x)$ 的值

方法一：算法驱动

- 只在需要 $f(x)$ 的值的时候，通过计算得到。

方法二：数据驱动

- 先通过计算，预存所有 x 对应的 $f(x)$ 值；需要计算的时候直接访问。

时间与空间的权衡 (trade-off)：以空间换时间，或以时间换空间。

案例 2: PhoneKeyPad

问题: 将字母字符串转为手机键盘数字

"AbcDEGj" → "2,2,2,3,3,4,5"

```
// 方法一: 算法驱动 (if-else 链)
if (c == 'a' || c == 'b' || c == 'c')
    result += "2";
else if (c == 'd' || c == 'e' || c == 'f')
    result += "3";
// ... (大量重复分支)
```

```
// 方法二: 数据驱动
private static String[] keyPad = {
    "", "", "ABC", "DEF",
    "GHI", "JKL", "MNO",
    "PQRS", "TUV", "WXYZ"
};
for (int j = 2; j < 10; j++) {
    if (keyPad[j].contains(c + ""))
        resultString += sp + j;
}
```

案例 3：消息处理

问题：处理不同类型的消息，每种类型调用对应的处理函数。

方法一：算法驱动

```
void msg_proc(const char *msg_type,
              const char *msg_buf) {
    if (strcmp(msg_type, "invite") == 0)
        invite_fun(msg_buf);
    else if (strcmp(msg_type, "ring_100") == 0)
        ring_100_fun(msg_buf);
    // ... 数十个 else-if 分支
}
```

每增加一种消息类型，必须修改 `msg_proc`。

方法二：数据驱动

```
typedef void (*SIP_MSG_FUN)(const char *);
typedef struct { const char *msg_type;
                 SIP_MSG_FUN fun_ptr; } msg_fun_st;

msg_fun_st msg_flow[] = {
    {"invite",  invite_fun},
    {"ring_100", ring_100_fun},
};

for (int i = 0; i < type_num; i++) {
    if (strcmp(msg_flow[i].msg_type, msg_type) == 0)
        msg_flow[i].fun_ptr(msg_buf);
}
```

新增消息类型只需在表格中加一行。

证明程序的正确性

How to Prove Your Program's Correctness

How to Prove — 两大方法

Edsger W. Dijkstra

- Enumeration (穷举/不变量)
- Mathematical Induction (数学归纳)
- Abstraction (抽象证明)

C. A. R. Hoare

- Axioms Proof (公理化证明)
- Hoare Triple: $\{P\} S \{Q\}$

How to Prove — Dijkstra

不变量方法示例 (计算 $a \bmod d$):

```
int r = a; int dd = d;
while (dd <= r) { dd = 2 * dd; }
while (dd != d) {
    dd = dd / 2;           // halve dd
    if (dd <= r) { r = r - dd; } // reduce r modulo dd
}
```

- 第一个 `while` 结束后: `dd > r`, 故**不变量**成立: `0 <= r < dd`
- 第二个 `while` 每次迭代: `dd` 减半, 若 `dd <= r` 则 `r -= dd`, 不变量**保持**
- 循环结束时 `dd == d`, 故 `0 <= r < d`, 即 `r = a mod d` ✓

How to Prove — Hoare 公理 (1/2)

Hoare Triple: $\{P\} S \{Q\}$ — 若前置条件 P 成立，执行语句 S 后，后置条件 Q 成立。

D0 赋值公理 (Assignment Axiom)

$$\vdash P_0 \{x := f\} P$$

其中 P_0 是将后置条件 P 中所有 x 替换为 f 所得 (反向替换)。

目标三元组	替换操作	P_0	成立?
$\{?\} y := x+1 \{y > x\}$	Q 中 $y \rightarrow x+1$	$x+1 > x$	✓ 恒真
$\{?\} x := x+1 \{x > 0\}$	Q 中 $x \rightarrow x+1$	$x+1 > 0$	需 $x \geq 0$
$\{?\} x := 5 \{x = 5\}$	Q 中 $x \rightarrow 5$	$5 = 5$	✓ 恒真

How to Prove — Hoare 公理 (2/2)

D1 推论规则: 若 $\vdash P\{Q\}R$ 且 $R \supset S$, 则 $\vdash P\{Q\}S$

(后置条件可以"弱化")

D2 复合规则: 若 $\vdash P\{Q_1\}R_1$ 且 $\vdash R_1\{Q_2\}R$, 则 $\vdash P\{Q_1; Q_2\}R$

D3 迭代规则: 若 $\vdash P \wedge B \{S\} P$, 则 $\vdash P\{\mathbf{while} B \mathbf{do} S\} \neg B \wedge P$

P 为循环不变量: 循环体执行一次后 P 仍成立; 退出时 $\neg B \wedge P$ 成立。

D2 示例: 证明 $\{x=a\} y:=x; x:=y+1 \{x=a+1\}$

- 步骤① $\{x=a\} y:=x \{y=a\}$ - D0: Q 中 $y \rightarrow x$, 得 $x=a$ ✓
- 步骤② $\{y=a\} x:=y+1 \{x=a+1\}$ - D0: Q 中 $x \rightarrow y+1$, 得 $y+1=a+1$, 即 $y=a$ ✓
- 步骤③ 由D2合并: $\{x=a\} y:=x; x:=y+1 \{x=a+1\}$ ✓

How to Prove — D3 迭代案例

目标：证明 $\{x \geq 0\} \text{ while } x > 0 \text{ do } x := x-1 \{x = 0\}$

选取循环不变量 P : $x \geq 0$

① 验证初始条件成立：前置条件 $x \geq 0 = P$ ✓

② 验证循环体保持不变量 (D0 + D2):

需证: $\{x \geq 0 \wedge x > 0\} \quad x := x-1 \quad \{x \geq 0\}$

D0: $Q \text{ 中 } x \rightarrow x-1$, 得 $x-1 \geq 0$, 即 $x \geq 1$

而前置 $x \geq 0 \wedge x > 0 \Rightarrow x \geq 1$ ✓

③ 应用 D3:

$P = x \geq 0$, $B = x > 0$

结论: $\{x \geq 0\} \text{ while } x > 0 \text{ do } x := x-1 \{ \neg(x > 0) \wedge x \geq 0 \}$

$= \{x \geq 0\} \dots \{x = 0 \wedge x \geq 0\}$

$\Rightarrow \{x \geq 0\} \dots \{x = 0\}$ ✓ (D1弱化)

三步固定模式：① 找不变量 → ② 验证循环体保持它 → ③ 用 D3 推出结论。

习题课

递归和迭代 · Lambda演算 · 过程建模

习题课 一 内容

- 递归和迭代：加法、Fibonacci、GCD、换零钱
- Lambda 演算：SUB FOUR TWO、IF (EQ ONE TWO) a b、LENGTH NIL
- 过程建模：accumulate、多项式求值 (horner-eval)、accumulate-n
- 高阶函数：map、filter、fold 的实现与应用
- 函数式编程：用 Java 8 Stream 重写命令式代码

递归和迭代

Recursion & Iteration

+ 加法

两种实现正整数加法的方式（基于 `inc` 和 `dec`）：

```
; 线性递归版本
(define (+ a b)
  (if (= a 0)
      b
      (inc (+ (dec a) b))))
```

```
; (+ 4 5)
; => (inc (+ 3 5))
; => (inc (inc (+ 2 5)))
; => ... (递归展开)
```

```
; 线性迭代版本
(define (+ a b)
  (if (= a 0)
      b
      (+ (dec a) (inc b))))
```

```
; (+ 4 5)
; => (+ 3 6)
; => (+ 2 7)
; => ... (尾递归)
```

Fibonacci 数列

$$\text{Fib}(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ \text{Fib}(n-1) + \text{Fib}(n-2) & \text{otherwise} \end{cases}$$

```
(define (fib n)
  (cond ((= n 0) 0)
        ((= n 1) 1)
        (else (+ (fib (- n 1))
                  (fib (- n 2))))))
```

这是树形递归，时间复杂度为 $O(\phi^n)$ 。如何改为迭代？

最大公约数

如果 r 是 a 除以 b 的余数，那么 a 和 b 的公约数也正好是 b 和 r 的公约数。

```
(define (gcd a b)
  (if (= b 0)
      a
      (gcd b (remainder a b))))
```

```
; (gcd 206 40)
; => (gcd 40 6)
; => (gcd 6 4)
; => (gcd 4 2)
; => (gcd 2 0)
; => 2
```

换零钱

将 1 美元换成半美元、四分之一美元、10 美分、5 美分、1 美分，共有多少种换法？

```
(define (count-change amount) (cc amount 5))

(define (cc amount kinds-of-coins)
  (cond ((= amount 0) 1)
        ((or (< amount 0) (= kinds-of-coins 0)) 0)
        (else (+ (cc amount (- kinds-of-coins 1))
                  (cc (- amount (first-denomination kinds-of-coins))
                      kinds-of-coins)))))

(define (first-denomination k)
  (cond ((= k 1) 1) ((= k 2) 5) ((= k 3) 10)
        ((= k 4) 25) ((= k 5) 50)))
```

Lambda 演算

Lambda Calculus

Lambda> SUB FOUR TWO

$SUCC = \lambda n. \lambda f. \lambda x. f (n f x)$

$PLUS = \lambda m. \lambda n. m SUCC n$

$PRED = \lambda n. \lambda f. \lambda x. n (\lambda g. \lambda h. h (g f)) (\lambda u. x) (\lambda u. u)$

$SUB = \lambda m. \lambda n. n PRED m$

求值 SUB FOUR TWO:

$SUB \text{ FOUR TWO}$

$= \lambda m. \lambda n. n PRED m \text{ FOUR TWO}$

$= TWO PRED FOUR$

$= \lambda f. \lambda x. f(f x) PRED FOUR$

$= PRED (PRED FOUR)$

$= TWO$

IF (EQ ONE TWO) a b

```
TRUE  =  $\lambda x.\lambda y. x$       FALSE =  $\lambda x.\lambda y. y$   
AND   =  $\lambda p.\lambda q. p\ q\ p$   OR    =  $\lambda p.\lambda q. p\ p\ q$   
NOT   =  $\lambda p.\lambda a.\lambda b. p\ b\ a$  IF   =  $\lambda p.\lambda a.\lambda b. p\ a\ b$   
ISZERO =  $\lambda n. n\ (\lambda x.FALSE)\ TRUE$   
LEQ   =  $\lambda m.\lambda n. ISZERO\ (SUB\ m\ n)$   
EQ    =  $\lambda m.\lambda n. AND\ (LEQ\ m\ n)\ (LEQ\ n\ m)$ 
```

求值 IF (EQ ONE TWO) a b:

EQ ONE TWO 经过化简得到 **FALSE**,
 $\therefore$ IF FALSE a b = $\lambda x.\lambda y. y\ a\ b = b$

LENGTH NIL

$Y = \lambda g. (\lambda x. g (x x)) \lambda x. g (x x)$ (Y 不动点组合子: $YF \Rightarrow_{\beta} F(YF)$)

$CONS = \lambda x. \lambda y. \lambda f. f x y$

$CAR = \lambda p. p \text{ TRUE}$

$CDR = \lambda p. p \text{ FALSE}$

$NIL = \lambda x. \text{ TRUE}$

$NULL = \lambda p. p (\lambda x. \lambda y. \text{ FALSE})$

$LENGTH = Y (\lambda g. \lambda c. \lambda x. NULL x c (g (SUCC c) (CDR x))) \text{ ZERO}$

$LENGTH \text{ NIL} = \text{ZERO} (= \lambda f. \lambda x. x)$

$LENGTH (CONS a \text{ NIL}) = \text{ONE}$

过程建模

Process Modelling

accumulate — 累积器

```
(define (accumulate op initial sequence)
  (if (null? sequence)
      initial
      (op (car sequence)
          (accumulate op initial (cdr sequence)))))

(accumulate + 0 (list 1 2 3 4 5)) ; => 15
(accumulate * 1 (list 1 2 3 4 5)) ; => 120
(accumulate cons nil (list 1 2 3 4 5)) ; => (1 2 3 4 5)
```

`accumulate` 是 `fold-right` 的别名，是信号流的累积器原语。

多项式求值 — Horner 法

$$\begin{aligned} & a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 \\ &= (\cdots (a_n x + a_{n-1}) x + \cdots + a_1) x + a_0 \end{aligned}$$

```
(define (horner-eval x coefficient-sequence)
  (accumulate
    (lambda (this-coeff higher-terms)
      (+ this-coeff (* x higher-terms)))
    0
    coefficient-sequence))
```

```
(horner-eval 2 '(1 3 0 5 0 1))
```

```
; = 1 + 2*3 + 4*0 + 8*5 + 16*0 + 32*1 = 79
```

accumulate-n

对多个序列按位置分别累积：

```
(define (accumulate-n op init seqs)
  (if (null? (car seqs))
      nil
      (cons (accumulate op init (map car seqs))
            (accumulate-n op init (map cdr seqs)))))

(accumulate-n + 0 '((1 2 3)
                    (4 5 6)
                    (7 8 9)
                    (10 11 12)))

; => (22 26 30)
```

【AI视角】函数式编程 × 现代 AI

Functional Programming Meets Machine Learning

【AI视角】JAX — 函数式机器学习框架

函数式核心思想

- **纯函数**：相同输入永远相同输出
- **不可变数组**：每次操作返回新张量
- **函数变换**：`grad`、`jit`、`vmap` 是高阶函数

```
import jax
import jax.numpy as jnp

# grad 是高阶函数：函数 → 梯度函数
grad_fn = jax.grad(lambda x: x**2)
grad_fn(3.0)  # => 6.0
```

与 SICP 概念的对应

JAX	函数式概念
<code>jax.grad(f)</code>	高阶函数
<code>jax.vmap(f)</code>	map（批量映射）
<code>jax.lax.scan</code>	信号流 accumulate
不可变 <code>Array</code>	无副作用

函数式编程不只是学术概念，它是现代深度学习框架（JAX、PyTorch 2.0）的设计基础。

【AI视角】LLM 的"推理"与函数式思维

LLM 生成代码的偏好

函数式风格更易生成正确代码：

- 纯函数无隐式状态，逻辑可局部验证
- `map` / `filter` / `reduce` 意图明确
- 递归结构与自然语言描述对应自然

```
# LLM 倾向生成这种风格
result = (
    list(map(lambda x: x**2,
             filter(lambda x: x % 2 == 1,
                    range(10))))
)
```

程序正确性 × AI 辅助验证

- **Hoare Logic**: 前后置条件可作为 Prompt 的规格说明
- **不变量**: 告诉 LLM "循环结束时应满足 X"
- **类型系统**: 函数签名是机器可检验的规格

Prompt 示例:

"写一个函数满足:

前置: $n \geq 0$

后置: 返回值 = $n!$

不变量: $acc * k! = n!$ 不变"

越精确的规格 = 越好的 AI 辅助效果。

总结

函数式编程范式 · 核心回顾

本讲总结 (1/2) — 核心概念

函数式核心

概念	要点
高阶函数	函数作为参数/返回值
数据抽象	构造器 + 选择器 + 抽象屏障
符号求导	代码即数据；列表即 AST
元编程	同像性 · 宏 · eval · 反射
信号流	枚举 → 过滤 → 映射 → 累积

程序正确性

方法	核心思想
Dijkstra	循环不变量，归纳证明
Hoare	$\{P\} S \{Q\}$ ； D0–D3 公理推导

元循环求值器

- `my-eval` 按表达式类型分发
- `eval` ↔ `apply` 相互递归驱动求值
- < 50 行代码实现一门完整语言

本讲总结 (2/2) — 思维转变

命令式 vs 函数式

命令式 (HOW): 逐步告诉计算机怎么做, 关注状态变化

sum-odd-squares: 枚举、过滤、映射、累积混杂在一起

函数式 (WHAT): 描述结果是什么, 关注变换组合

sum-odd-squares: `enumerate` → `filter odd?` → `map square` → `accumulate` +

每步单一职责, 可独立测试和复用

一句话

函数式编程的本质: 用函数组合替代状态变化,
用声明描述"是什么"而非命令描述"怎么做"。

知识延伸

- 元编程 → SICP 第四章: 元循环求值器、惰性求值、非确定性计算
- 信号流 → 现代 ML 框架 (JAX、PyTorch 2.0) 的设计基础
- Hoare Logic → 形式化验证 (Coq、Isabelle、Lean)